

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Case Study

COURSE NAME: Query Processing for Data Science **COURSE CODE:** DSA0503

COURSE OUTCOME COVERED

CO4: Explore datasets using analytical techniques such as grouping, correlation detection, joining datasets, and extracting meaningful insights.

Assessment Tool Weightage: 35%

Total Marks: 20

Code of Conduct

I NARESH M(192424381) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Naresh M

Case Study Rubric

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided

Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand
---------	----	--	---	-------------------------------	---------------------------------------

Total Marks Awarded:

Signature of the Course Faculty

Questions:

SET 3

1. Transportation and Traffic Analysis

A city transportation department maintains data containing vehicle type, route, date, travel time, distance, traffic volume, and location. Explore the dataset, create groups based on vehicle type and route, identify outliers in travel time and traffic volume, and analyze relationships between distance, traffic volume, and travel time. Use time-related visualizations to identify traffic patterns and, where geographical information is available, represent important findings using maps. Prepare a report explaining how the results can support traffic management.

Aim

To analyze transportation and traffic data using Pandas and Matplotlib and identify traffic patterns, outliers and relationships among travel variables.

Code:

```
import pandas as pd

import matplotlib.pyplot as plt

data = {
    "Vehicle_Type": ["Car","Bus","Bike","Truck","Car","Bus","Bike","Truck","Car","Bus","Bike","Truck"],
    "Route": ["R1","R1","R2","R2","R3","R3","R1","R1","R2","R2","R3","R3"],
    "Date": ["2026-01-01","2026-01-02","2026-01-03","2026-01-04",
            "2026-01-05","2026-01-06","2026-01-07","2026-01-08",
            "2026-01-09","2026-01-10","2026-01-11","2026-01-12"],
    "Travel_Time": [25,40,18,55,30,45,20,90,35,50,22,60],
    "Distance": [10,15,8,20,12,18,9,22,14,17,10,21],
```

```

    "Traffic_Volume": [500,700,300,900,550,800,350,1500,600,850,400,950],
    "Location": ["North","North","East","East","South","South",
                 "North","North","East","East","South","South"]
}

df = pd.DataFrame(data)

df["Date"] = pd.to_datetime(df["Date"])

print(df)

print(df.info())

print(df.describe())

print(df.isnull().sum())

vehicle_group = df.groupby("Vehicle_Type")[["Travel_Time","Distance","Traffic_Volume"]].mean()

route_group = df.groupby("Route")[["Travel_Time","Distance","Traffic_Volume"]].mean()

print(vehicle_group)

print(route_group)

correlation = df[["Distance","Traffic_Volume","Travel_Time"]].corr()

print(correlation)

Q1 = df["Travel_Time"].quantile(0.25)

Q3 = df["Travel_Time"].quantile(0.75)

IQR = Q3 - Q1

outliers = df[(df["Travel_Time"] < Q1 - 1.5 * IQR) |
              (df["Travel_Time"] > Q3 + 1.5 * IQR)]

print(outliers)

plt.figure(figsize=(8,5))

plt.plot(df["Date"], df["Travel_Time"], marker="o")

plt.xlabel("Date")

plt.ylabel("Travel Time")

plt.title("Travel Time Over Time")

plt.xticks(rotation=45)

plt.tight_layout()

plt.show()

plt.figure(figsize=(7,5))

```

```
plt.scatter(df["Distance"], df["Travel_Time"])

plt.xlabel("Distance")

plt.ylabel("Travel Time")

plt.title("Distance vs Travel Time")

plt.show()

plt.figure(figsize=(7,5))

plt.scatter(df["Traffic_Volume"], df["Travel_Time"])

plt.xlabel("Traffic Volume")

plt.ylabel("Travel Time")

plt.title("Traffic Volume vs Travel Time")

plt.show()
```

output:

```
===== RESTART: C:/Users/nares/Videos/dsa0503/experiments/ass 4.1.1.py =====
  Vehicle_Type Route      Date  ... Distance Traffic_Volume Location
0         Car   R1 2026-01-01  ...      10         500      North
1         Bus   R1 2026-01-02  ...      15         700      North
2        Bike   R2 2026-01-03  ...       8         300       East
3        Truck  R2 2026-01-04  ...      20         900       East
4         Car   R3 2026-01-05  ...      12         550      South
5         Bus   R3 2026-01-06  ...      18         800      South
6        Bike   R1 2026-01-07  ...       9         350      North
7        Truck  R1 2026-01-08  ...      22        1500      North
8         Car   R2 2026-01-09  ...      14         600       East
9         Bus   R2 2026-01-10  ...      17         850       East
10        Bike  R3 2026-01-11  ...      10         400      South
11        Truck  R3 2026-01-12  ...      21         950      South

[12 rows x 7 columns]
<class 'pandas.DataFrame'>
RangeIndex: 12 entries, 0 to 11
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Vehicle_Type          12 non-null    str
1   Route                 12 non-null    str
2   Date                  12 non-null    datetime64[us]
3   Travel_Time           12 non-null    int64
4   Distance              12 non-null    int64
5   Traffic_Volume        12 non-null    int64
6   Location              12 non-null    str
dtypes: datetime64[us](1), int64(3), str(3)
memory usage: 804.0 bytes
None
```

```

count          12      12.000000  12.000000  12.000000
mean  2026-01-06 12:00:00  40.833333  14.666667  700.000000
min    2026-01-01 00:00:00  18.000000   8.000000  300.000000
25%    2026-01-03 18:00:00  24.250000  10.000000  475.000000
50%    2026-01-06 12:00:00  37.500000  14.500000  650.000000
75%    2026-01-09 06:00:00  51.250000  18.500000  862.500000
max    2026-01-12 00:00:00  90.000000  22.000000 1500.000000
std              NaN    20.888593   4.923660  333.030165
Vehicle_Type    0
Route           0
Date            0
Travel_Time     0
Distance        0
Traffic_Volume  0
Location        0
dtype: int64

```

```

      Travel_Time  Distance  Traffic_Volume
Vehicle_Type
Bike           20.000000    9.000000    350.000000
Bus            45.000000   16.666667    783.333333
Car            30.000000   12.000000    550.000000
Truck          68.333333   21.000000   1116.666667

```

```

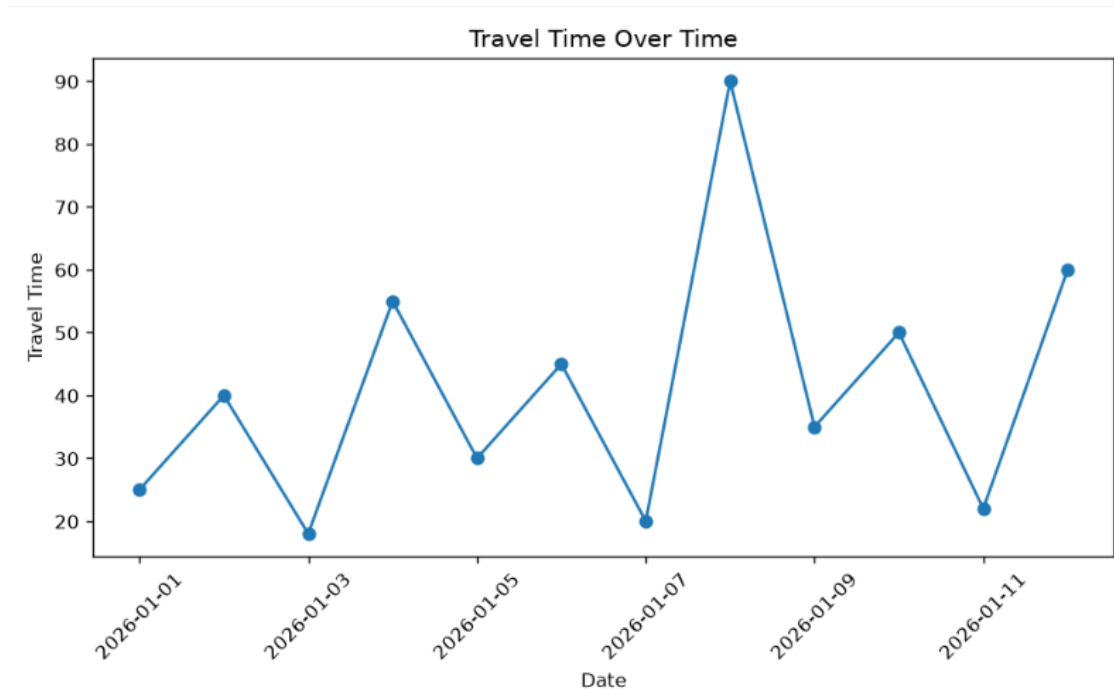
      Travel_Time  Distance  Traffic_Volume
Route
R1           43.75    14.00         762.5
R2           39.50    14.75         662.5
R3           39.25    15.25         675.0
      Distance  Traffic_Volume  Travel_Time
Distance      1.000000      0.923103    0.934592
Traffic_Volume 0.923103      1.000000    0.995794
Travel_Time    0.934592      0.995794    1.000000

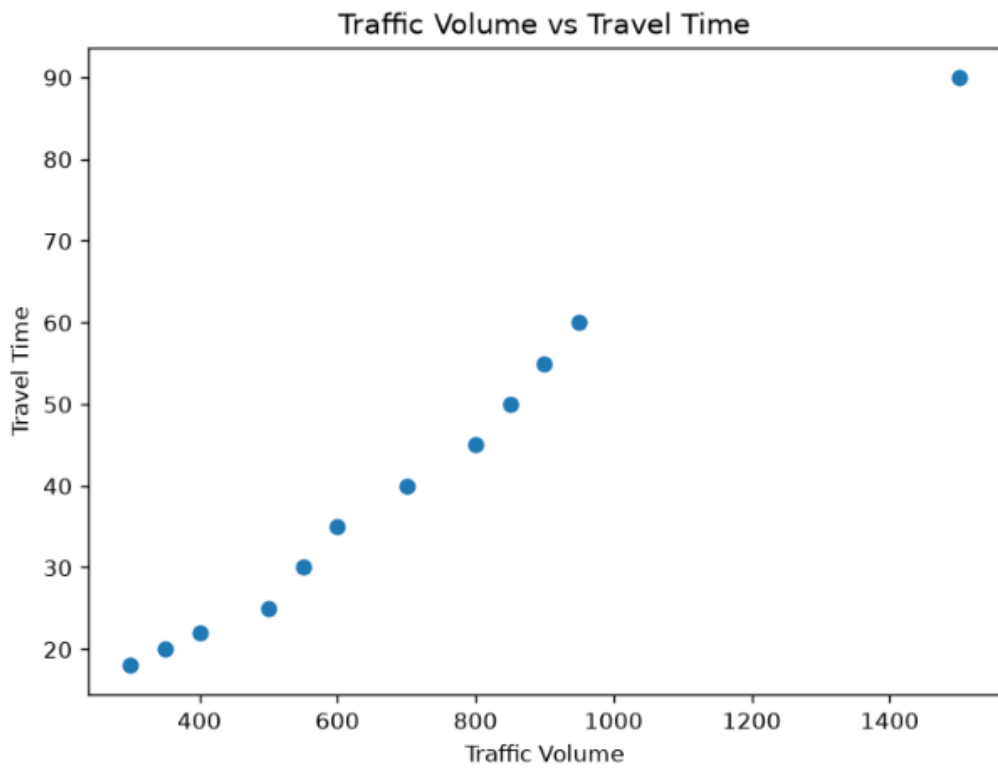
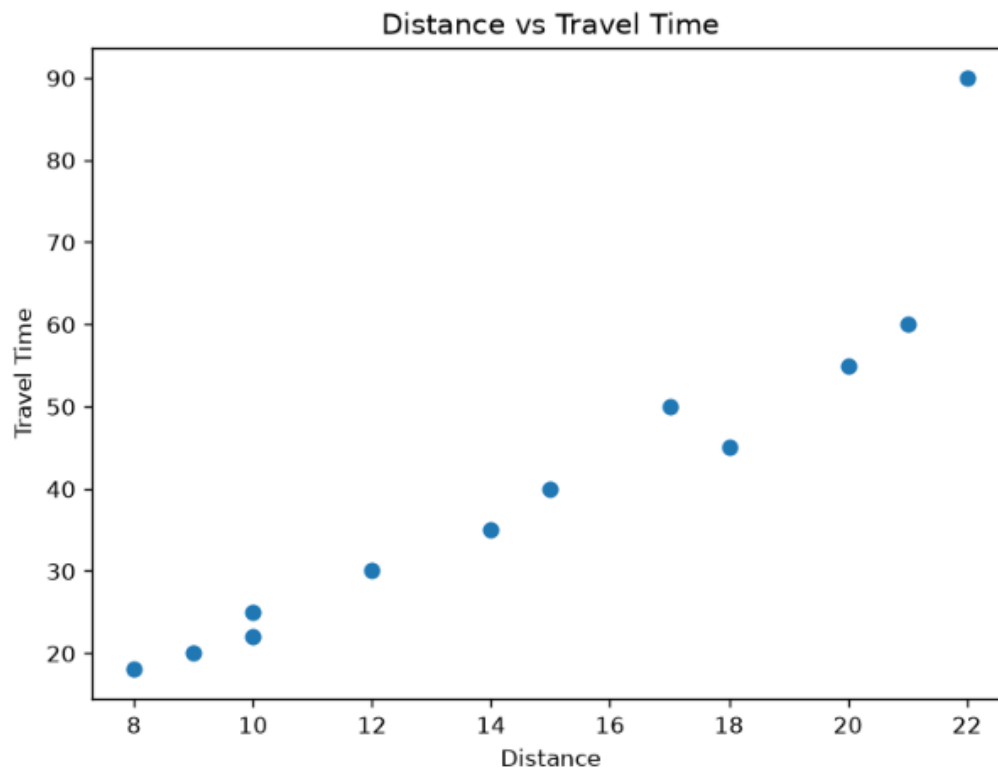
```

```
Empty DataFrame
```

```
Columns: [Vehicle_Type, Route, Date, Travel_Time, Distance, Traffic_Volume, Location]
```

```
Index: []
```





Result

The transportation dataset was successfully explored and analyzed. Grouping, correlation analysis, outlier detection and time-based visualization helped identify traffic patterns that can support route planning, congestion management and transportation scheduling.

2. Food Delivery Analysis

A food delivery platform has collected order information including customer location, restaurant, food category, order value, delivery time, order date, and customer rating. Analyze the dataset by exploring its structure, grouping orders by restaurant and food category, identifying unusual delivery times, and determining whether order value is related to customer ratings or delivery time. Analyze changes in orders over time and present the important findings using charts and other suitable visualizations.

Aim

To analyze food delivery orders and identify customer ordering patterns, delivery performance and relationships among order value, delivery time and customer ratings.

Code:

```
import pandas as pd

import matplotlib.pyplot as plt

data = {
    "Customer_Location": ["Chennai","Chennai","Bangalore","Chennai","Bangalore",
                          "Hyderabad","Chennai","Hyderabad","Bangalore","Chennai",
                          "Hyderabad","Bangalore"],
    "Restaurant": ["A2B","Dominos","Empire","A2B","Dominos","Paradise",
                  "Empire","Paradise","A2B","Dominos","Empire","Paradise"],
    "Food_Category": ["South Indian","Pizza","Biryani","South Indian","Pizza",
                     "Biryani","Chinese","Biryani","South Indian","Pizza",
                     "Chinese","Biryani"],
    "Order_Value": [250,450,600,300,500,700,350,800,280,550,400,750],
    "Delivery_Time": [30,40,50,35,45,60,38,75,28,55,42,65],
    "Order_Date": ["2026-01-01","2026-01-02","2026-01-03","2026-01-04",
                  "2026-01-05","2026-01-06","2026-01-07","2026-01-08",
                  "2026-01-09","2026-01-10","2026-01-11","2026-01-12"],
    "Customer_Rating": [4.5,4.2,4.0,4.6,4.1,3.5,4.3,3.2,4.7,3.9,4.4,3.4]
}

df = pd.DataFrame(data)

df["Order_Date"] = pd.to_datetime(df["Order_Date"])

print(df)

print(df.info())

print(df.describe())
```

```

print(df.isnull().sum())

restaurant_group = df.groupby("Restaurant").agg(
    Orders=("Restaurant","count"),
    Average_Order_Value=("Order_Value","mean"),
    Average_Delivery_Time=("Delivery_Time","mean"),
    Average_Rating=("Customer_Rating","mean")
)

print(restaurant_group)

category_group = df.groupby("Food_Category").agg(
    Orders=("Food_Category","count"),
    Average_Order_Value=("Order_Value","mean"),
    Average_Delivery_Time=("Delivery_Time","mean")
)

print(category_group)

correlation = df[["Order_Value","Delivery_Time","Customer_Rating"]].corr()

print(correlation)

Q1 = df["Delivery_Time"].quantile(0.25)
Q3 = df["Delivery_Time"].quantile(0.75)
IQR = Q3 - Q1

outliers = df[(df["Delivery_Time"] < Q1 - 1.5 * IQR) |
               (df["Delivery_Time"] > Q3 + 1.5 * IQR)]

print(outliers)

plt.figure(figsize=(8,5))

plt.plot(df["Order_Date"], df["Order_Value"], marker="o")

plt.xlabel("Order Date")

plt.ylabel("Order Value")

plt.title("Order Value Over Time")

plt.xticks(rotation=45)

plt.tight_layout()

plt.show()

plt.figure(figsize=(7,5))

```

```
plt.scatter(df["Delivery_Time"], df["Customer_Rating"])

plt.xlabel("Delivery Time")

plt.ylabel("Customer Rating")

plt.title("Delivery Time vs Customer Rating")

plt.show()

plt.figure(figsize=(7,5))

plt.scatter(df["Order_Value"], df["Customer_Rating"])

plt.xlabel("Order Value")

plt.ylabel("Customer Rating")

plt.title("Order Value vs Customer Rating")

plt.show()
```

output:

```
===== RESTART: C:/Users/nares/Videos/dsa1
Customer_Location Restaurant ... Order_Date Customer_Rating
0 Chennai A2B ... 2026-01-01 4.5
1 Chennai Dominos ... 2026-01-02 4.2
2 Bangalore Empire ... 2026-01-03 4.0
3 Chennai A2B ... 2026-01-04 4.6
4 Bangalore Dominos ... 2026-01-05 4.1
5 Hyderabad Paradise ... 2026-01-06 3.5
6 Chennai Empire ... 2026-01-07 4.3
7 Hyderabad Paradise ... 2026-01-08 3.2
8 Bangalore A2B ... 2026-01-09 4.7
9 Chennai Dominos ... 2026-01-10 3.9
10 Hyderabad Empire ... 2026-01-11 4.4
11 Bangalore Paradise ... 2026-01-12 3.4

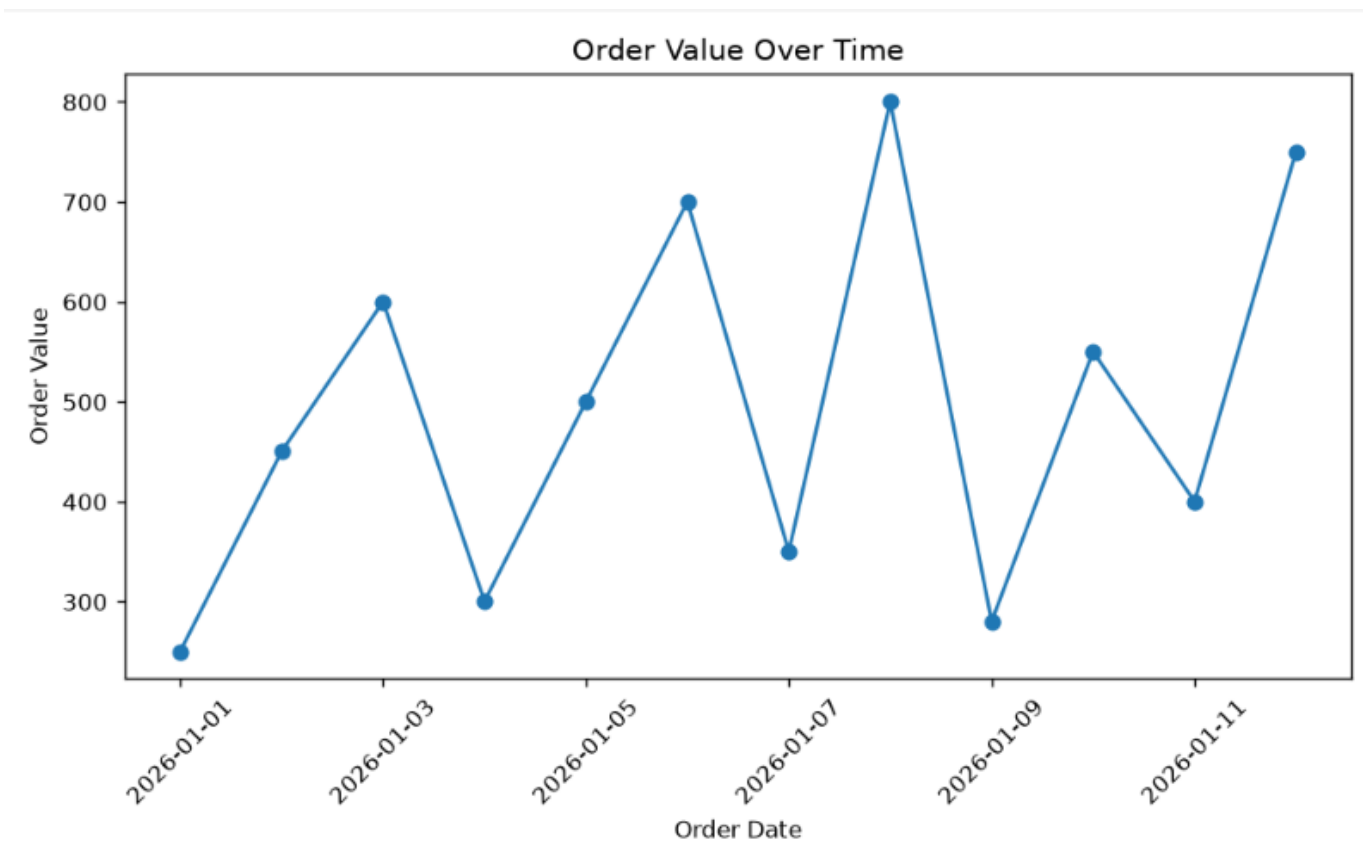
[12 rows x 7 columns]
<class 'pandas.DataFrame'>
RangeIndex: 12 entries, 0 to 11
Data columns (total 7 columns):
# Column Non-Null Count Dtype
---
0 Customer_Location 12 non-null str
1 Restaurant 12 non-null str
2 Food_Category 12 non-null str
3 Order_Value 12 non-null int64
4 Delivery_Time 12 non-null int64
5 Order_Date 12 non-null datetime64[us]
6 Customer_Rating 12 non-null float64
dtypes: datetime64[us](1), float64(1), int64(2), str(3)
memory usage: 804.0 bytes
None
```

```

count      Order_Value  Delivery_Time      Order_Date  Customer_Rating
mean      494.166667    46.916667    2026-01-06 12:00:00    4.066667
min       250.000000    28.000000    2026-01-01 00:00:00    3.200000
25%       337.500000    37.250000    2026-01-03 18:00:00    3.800000
50%       475.000000    43.500000    2026-01-06 12:00:00    4.150000
75%       625.000000    56.250000    2026-01-09 06:00:00    4.425000
max       800.000000    75.000000    2026-01-12 00:00:00    4.700000
std       188.412426    14.469142                NaN        0.486795
Customer_Location      0
Restaurant              0
Food_Category          0
Order_Value            0
Delivery_Time          0
Order_Date             0
Customer_Rating        0
dtype: int64

Orders  Average_Order_Value  Average_Delivery_Time  Average_Rating
Restaurant
A2B      3          276.666667          31.000000          4.600000
Dominos  3          500.000000          46.666667          4.066667
Empire   3          450.000000          43.333333          4.233333
Paradise  3          750.000000          66.666667          3.366667
Food_Category
Biriyani  4          712.500000          62.500000
Chinese   2          375.000000          40.000000
Pizza     3          500.000000          46.666667
South Indian  3          276.666667          31.000000
Order_Value  Delivery_Time  Customer_Rating
Order_Value  1.000000      0.974534      -0.975649
Delivery_Time 0.974534      1.000000      -0.977474
Customer_Rating -0.975649 -0.977474      1.000000
Empty DataFrame
Columns: [Customer_Location, Restaurant, Food_Category, Order_Value, Delivery_Time, Order_Date, Customer_Rating]
Index: []

```





Result

The food delivery data was successfully analyzed using grouping, correlation, outlier detection and visualization. The analysis can help the platform improve delivery efficiency, identify poorly performing restaurants and improve customer satisfaction.

3. Movie and Entertainment Analysis

A streaming platform has collected information about movies and shows, including title, genre, release year, duration, language, rating, number of viewers, and country. Explore and analyze the dataset, create groups based on genre and language, identify relationships between ratings, duration, and viewership, and detect unusual values. Present trends across release years using time-related charts and use appropriate visualizations to identify the most popular genres and content categories.

Aim

To analyze movie and entertainment data and identify popular genres, languages, viewing patterns and relationships between rating, duration and viewership.

Code:

```
import pandas as pd

import matplotlib.pyplot as plt

data = {
    "Title": ["Movie A", "Movie B", "Movie C", "Movie D", "Movie E", "Movie F",
             "Movie G", "Movie H", "Movie I", "Movie J", "Movie K", "Movie L"],
    "Genre": ["Action", "Comedy", "Drama", "Action", "Thriller", "Comedy",
             "Drama", "Action", "Thriller", "Comedy", "Drama", "Action"],
    "Release_Year": [2018, 2019, 2020, 2021, 2022, 2023, 2019, 2020, 2021, 2022, 2023, 2024],
    "Duration": [120, 100, 135, 150, 140, 95, 130, 160, 125, 105, 145, 155],
    "Language": ["English", "Tamil", "Hindi", "English", "Tamil", "Hindi",
                "English", "Tamil", "Hindi", "English", "Tamil", "Hindi"],
    "Rating": [8.1, 7.2, 8.5, 8.8, 7.9, 7.0, 8.2, 9.0, 7.8, 7.4, 8.6, 8.9],
    "Viewers": [500000, 300000, 650000, 900000, 550000, 250000,
               600000, 1000000, 450000, 350000, 750000, 950000],
    "Country": ["USA", "India", "India", "USA", "India", "India",
               "USA", "India", "India", "USA", "India", "India"]
}

df = pd.DataFrame(data)

print(df)

print(df.info())

print(df.describe())

print(df.isnull().sum())

genre_group = df.groupby("Genre").agg(
```

```

Movies=("Title","count"),
Average_Rating=("Rating","mean"),
Average_Viewers=("Viewers","mean"),
Average_Duration=("Duration","mean")
)
print(genre_group)
language_group = df.groupby("Language").agg(
    Movies=("Title","count"),
    Average_Rating=("Rating","mean"),
    Average_Viewers=("Viewers","mean")
)
print(language_group)
correlation = df[["Rating","Duration","Viewers"]].corr()
print(correlation)
Q1 = df["Viewers"].quantile(0.25)
Q3 = df["Viewers"].quantile(0.75)
IQR = Q3 - Q1
outliers = df[(df["Viewers"] < Q1 - 1.5 * IQR) |
               (df["Viewers"] > Q3 + 1.5 * IQR)]
print(outliers)
year_group = df.groupby("Release_Year")["Viewers"].sum()
print(year_group)
plt.figure(figsize=(8,5))
plt.plot(year_group.index, year_group.values, marker="o")
plt.xlabel("Release Year")
plt.ylabel("Total Viewers")
plt.title("Viewership Trend by Release Year")
plt.show()
plt.figure(figsize=(8,5))
genre_group["Average_Viewers"].plot(kind="bar")
plt.xlabel("Genre")

```

```
plt.ylabel("Average Viewers")

plt.title("Average Viewership by Genre")

plt.xticks(rotation=0)

plt.show()

plt.figure(figsize=(7,5))

plt.scatter(df["Rating"], df["Viewers"])

plt.xlabel("Rating")

plt.ylabel("Viewers")

plt.title("Rating vs Viewership")

plt.show()

plt.figure(figsize=(7,5))

plt.scatter(df["Duration"], df["Viewers"])

plt.xlabel("Duration")

plt.ylabel("Viewers")

plt.title("Duration vs Viewership")

plt.show()
```

Output:

```
===== RESTART: C:/Users/nares/
   Title  Genre  Release_Year  Duration  Language  Rating  Viewers  Country
0  Movie A  Action         2018        120  English    8.1    500000    USA
1  Movie B  Comedy         2019         100   Tamil     7.2    300000   India
2  Movie C  Drama          2020         135   Hindi     8.5    650000   India
3  Movie D  Action         2021         150  English    8.8    900000    USA
4  Movie E  Thriller        2022         140   Tamil     7.9    550000   India
5  Movie F  Comedy         2023          95   Hindi     7.0    250000   India
6  Movie G  Drama          2019         130  English    8.2    600000    USA
7  Movie H  Action         2020         160   Tamil     9.0   1000000   India
8  Movie I  Thriller        2021         125   Hindi     7.8    450000   India
9  Movie J  Comedy         2022         105  English    7.4    350000    USA
10 Movie K  Drama          2023         145   Tamil     8.6    750000   India
11 Movie L  Action         2024         155   Hindi     8.9    950000   India
<class 'pandas.DataFrame'>
RangeIndex: 12 entries, 0 to 11
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Title           12 non-null    str
1   Genre           12 non-null    str
2   Release_Year    12 non-null    int64
3   Duration        12 non-null    int64
4   Language        12 non-null    str
5   Rating          12 non-null    float64
6   Viewers         12 non-null    int64
7   Country         12 non-null    str
dtypes: float64(1), int64(3), str(4)
memory usage: 900.0 bytes
None
```

```

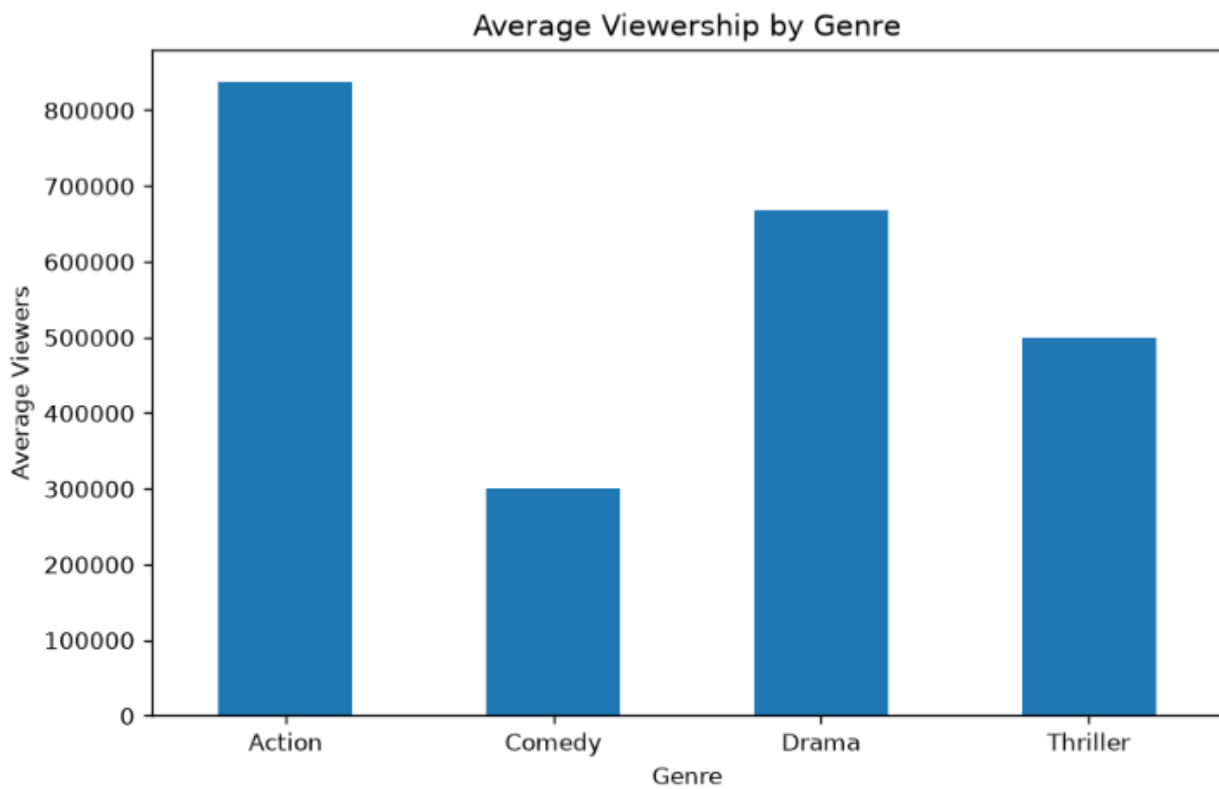
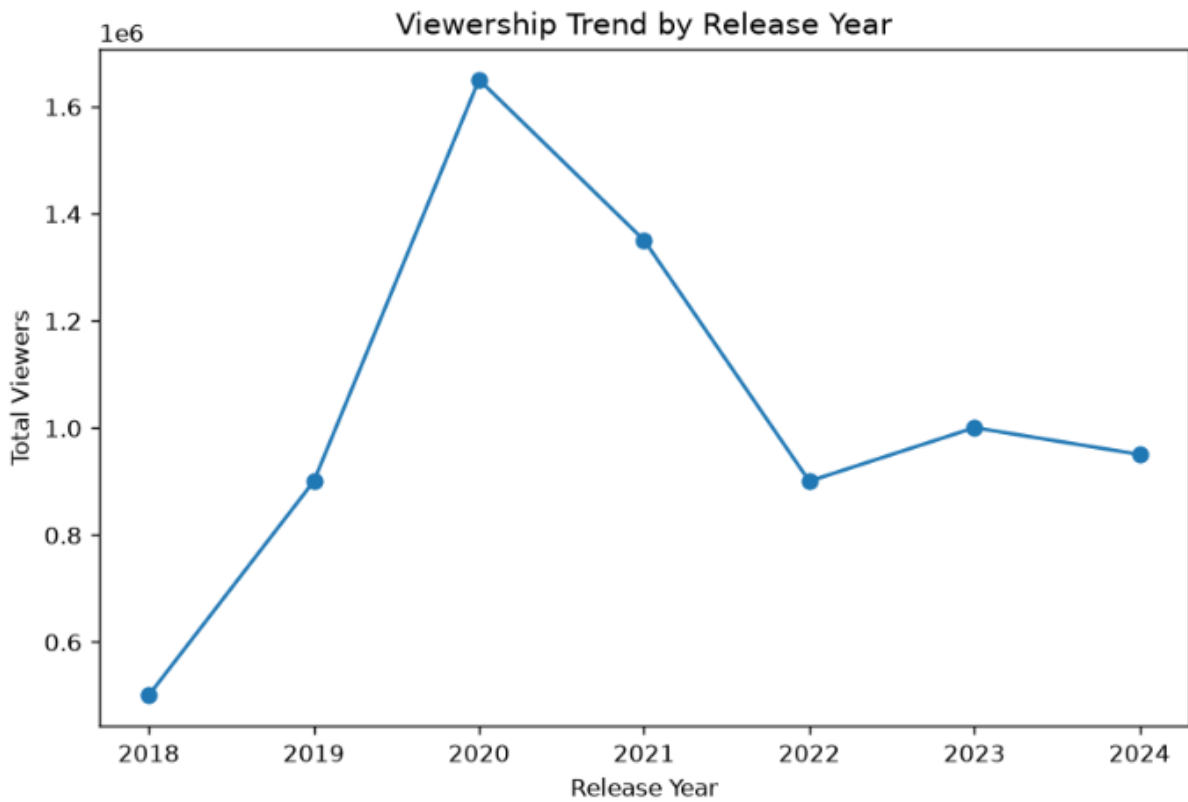
count      Release_Year  Duration  Rating  Viewers
mean      2021.000000  130.00000  8.116667  604166.666667
std       1.858641    21.63751  0.673975  253572.952864
min       2018.000000  95.00000  7.000000  250000.000000
25%       2019.750000  116.25000  7.700000  425000.000000
50%       2021.000000  132.50000  8.150000  575000.000000
75%       2022.250000  146.25000  8.650000  787500.000000
max       2024.000000  160.00000  9.000000  1000000.000000
Title      0
Genre      0
Release_Year  0
Duration    0
Language    0
Rating      0
Viewers     0
Country     0
dtype: int64

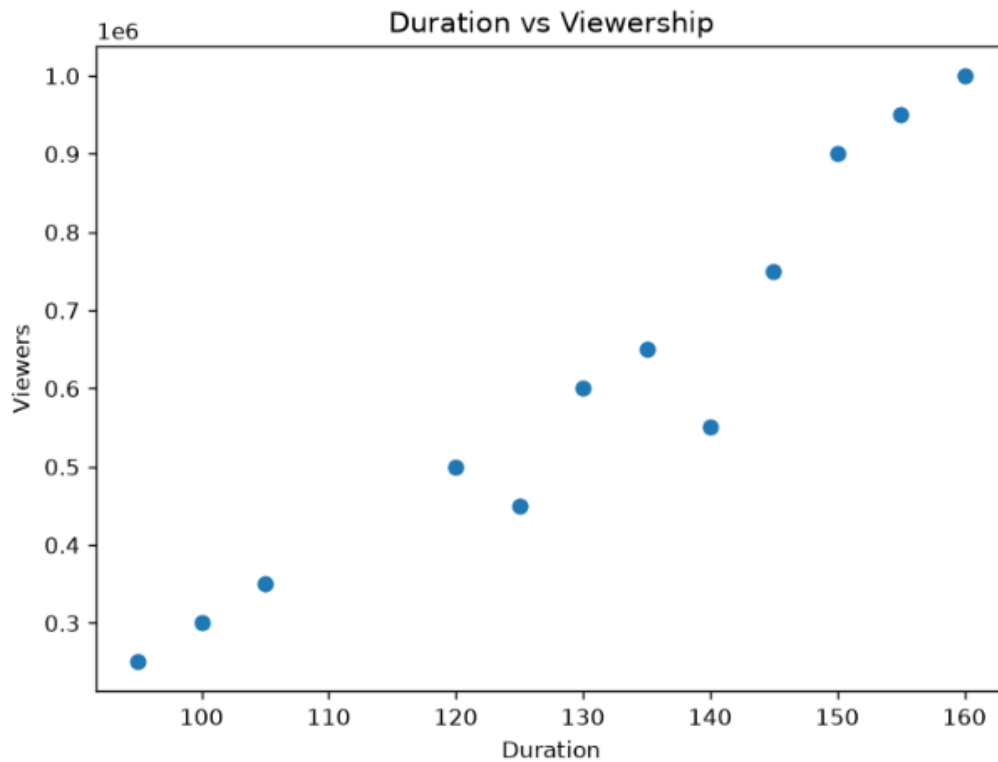
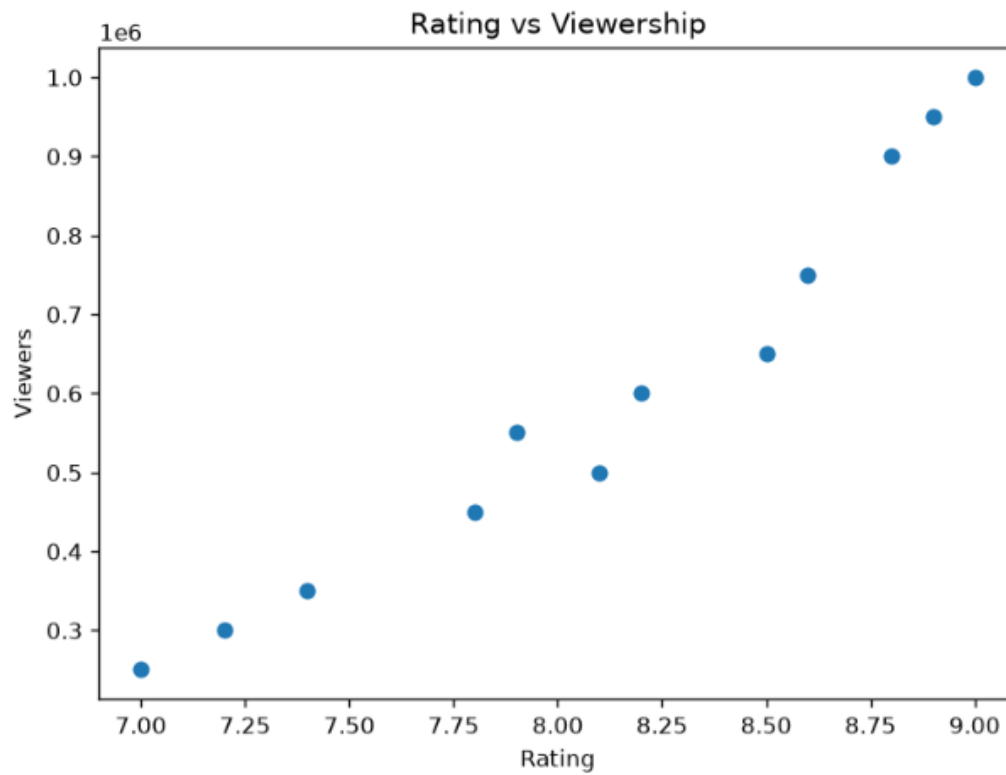
      Movies  Average_Rating  Average_Viewers  Average_Duration
Genre
Action      4           8.700000    837500.000000    146.250000
Comedy      3           7.200000    300000.000000    100.000000
Drama      3           8.433333    666666.666667    136.666667
Thriller    2           7.850000    500000.000000    132.500000

      Movies  Average_Rating  Average_Viewers
Language
English     4           8.125    587500.0
Hindi       4           8.050    575000.0
Tamil       4           8.175    650000.0

      Rating  Duration  Viewers
Rating    1.000000  0.953779  0.973002
Duration  0.953779  1.000000  0.961004
Viewers   0.973002  0.961004  1.000000
Empty DataFrame
Columns: [Title, Genre, Release_Year, Duration, Language, Rating, Viewers, Country]
Index: []
Release_Year
2018    500000
2019    900000
2020   1650000
2021   1350000
2022    900000
2023   1000000
2024    950000
Name: Viewers, dtype: int64

```





Result

The movie dataset was successfully analyzed using grouping, correlation, outlier detection and time-series visualization. The findings can help a streaming platform identify popular genres, understand viewer preferences and plan future content acquisition.

4. Agriculture and Crop Analysis

An agricultural department has collected data containing crop type, district, cultivated area, rainfall, fertilizer usage, and crop yield. Import and explore the data, group the records by crop and district, identify outliers in yield and fertilizer usage, and analyze correlations between rainfall, cultivated area, fertilizer usage, and crop yield. Use charts and maps, where geographical information is available, to communicate the major findings and suggest suitable strategies for improving agricultural productivity.

Aim

To analyze agricultural data and identify relationships between rainfall, cultivated area, fertilizer usage and crop yield.

Code:

```
import pandas as pd

import matplotlib.pyplot as plt

data = {

    "Crop_Type": ["Rice", "Wheat", "Rice", "Maize", "Wheat", "Rice",

                  "Maize", "Rice", "Wheat", "Maize", "Rice", "Wheat"],

    "District": ["Chennai", "Salem", "Madurai", "Coimbatore", "Salem", "Chennai",

                 "Madurai", "Coimbatore", "Chennai", "Salem", "Madurai", "Coimbatore"],

    "Cultivated_Area": [500, 450, 600, 400, 550, 700, 350, 650, 480, 420, 620, 580],

    "Rainfall": [1200, 800, 1100, 900, 850, 1300, 1000, 1150, 780, 880, 1080, 920],

    "Fertilizer_Usage": [250, 220, 280, 190, 240, 300, 180, 290, 210, 200, 270, 250],

    "Crop_Yield": [1800, 1500, 2100, 1300, 1700, 2400, 1200, 2300, 1600, 1400, 2200, 1850]

}

df = pd.DataFrame(data)

print(df)

print(df.info())

print(df.describe())

print(df.isnull().sum())

crop_group = df.groupby("Crop_Type").agg(

    Records=("Crop_Type", "count"),

    Average_Area=("Cultivated_Area", "mean"),

    Average_Rainfall=("Rainfall", "mean"),

    Average_Fertilizer=("Fertilizer_Usage", "mean"),
```

```

    Average_Yield=("Crop_Yield","mean")
)
print(crop_group)

district_group = df.groupby("District").agg(
    Average_Area=("Cultivated_Area","mean"),
    Average_Rainfall=("Rainfall","mean"),
    Average_Fertilizer=("Fertilizer_Usage","mean"),
    Average_Yield=("Crop_Yield","mean")
)

print(district_group)

correlation = df[["Rainfall","Cultivated_Area",
    "Fertilizer_Usage","Crop_Yield"]].corr()

print(correlation)

Q1 = df["Crop_Yield"].quantile(0.25)
Q3 = df["Crop_Yield"].quantile(0.75)
IQR = Q3 - Q1
yield_outliers = df[(df["Crop_Yield"] < Q1 - 1.5 * IQR) |
    (df["Crop_Yield"] > Q3 + 1.5 * IQR)]

print(yield_outliers)

Q1 = df["Fertilizer_Usage"].quantile(0.25)
Q3 = df["Fertilizer_Usage"].quantile(0.75)
IQR = Q3 - Q1
fertilizer_outliers = df[(df["Fertilizer_Usage"] < Q1 - 1.5 * IQR) |
    (df["Fertilizer_Usage"] > Q3 + 1.5 * IQR)]

print(fertilizer_outliers)

plt.figure(figsize=(8,5))

crop_group["Average_Yield"].plot(kind="bar")

plt.xlabel("Crop Type")

plt.ylabel("Average Crop Yield")

```

```
plt.title("Average Crop Yield by Crop Type")

plt.xticks(rotation=0)

plt.show()

plt.figure(figsize=(7,5))

plt.scatter(df["Rainfall"], df["Crop_Yield"])

plt.xlabel("Rainfall")

plt.ylabel("Crop Yield")

plt.title("Rainfall vs Crop Yield")

plt.show()

plt.figure(figsize=(7,5))

plt.scatter(df["Fertilizer_Usage"], df["Crop_Yield"])

plt.xlabel("Fertilizer Usage")

plt.ylabel("Crop Yield")

plt.title("Fertilizer Usage vs Crop Yield")

plt.show()

plt.figure(figsize=(7,5))

plt.scatter(df["Cultivated_Area"], df["Crop_Yield"])

plt.xlabel("Cultivated Area")

plt.ylabel("Crop Yield")

plt.title("Cultivated Area vs Crop Yield")

plt.show()
```

Output:

===== RESTART:

	Crop_Type	District	...	Fertilizer_Usage	Crop_Yield
0	Rice	Chennai	...	250	1800
1	Wheat	Salem	...	220	1500
2	Rice	Madurai	...	280	2100
3	Maize	Coimbatore	...	190	1300
4	Wheat	Salem	...	240	1700
5	Rice	Chennai	...	300	2400
6	Maize	Madurai	...	180	1200
7	Rice	Coimbatore	...	290	2300
8	Wheat	Chennai	...	210	1600
9	Maize	Salem	...	200	1400
10	Rice	Madurai	...	270	2200
11	Wheat	Coimbatore	...	250	1850

[12 rows x 6 columns]

<class 'pandas.DataFrame'>

RangeIndex: 12 entries, 0 to 11

Data columns (total 6 columns):

#	Column	Non-Null Count	Dtype
0	Crop_Type	12 non-null	str
1	District	12 non-null	str
2	Cultivated_Area	12 non-null	int64
3	Rainfall	12 non-null	int64
4	Fertilizer_Usage	12 non-null	int64
5	Crop_Yield	12 non-null	int64

dtypes: int64(4), str(2)

memory usage: 708.0 bytes

None

	Cultivated_Area	Rainfall	Fertilizer_Usage	Crop_Yield
count	12.000000	12.000000	12.000000	12.000000
mean	525.000000	996.666667	240.000000	1779.166667
std	108.753265	168.000721	40.226631	400.827742
min	350.000000	780.000000	180.000000	1200.000000
25%	442.500000	872.500000	207.500000	1475.000000
50%	525.000000	960.000000	245.000000	1750.000000
75%	605.000000	1112.500000	272.500000	2125.000000
max	700.000000	1300.000000	300.000000	2400.000000

Crop_Type	0
District	0
Cultivated_Area	0
Rainfall	0
Fertilizer_Usage	0
Crop_Yield	0

dtype: int64

	Records	Average_Area	...	Average_Fertilizer	Average_Yield
Crop_Type			...		
Maize	3	390.0	...	190.0	1300.0
Rice	5	614.0	...	278.0	2160.0
Wheat	4	515.0	...	230.0	1662.5

[3 rows x 5 columns]

	Average_Area	Average_Rainfall	Average_Fertilizer	Average_Yield
District				
Chennai	560.000000	1093.333333	253.333333	1933.333333
Coimbatore	543.333333	990.000000	243.333333	1816.666667
Madurai	523.333333	1060.000000	243.333333	1833.333333
Salem	473.333333	843.333333	220.000000	1533.333333

	Rainfall	Cultivated_Area	Fertilizer_Usage	Crop_Yield
Rainfall	1.000000	0.613006	0.719675	0.714383
Cultivated_Area	0.613006	1.000000	0.970439	0.978614
Fertilizer_Usage	0.719675	0.970439	1.000000	0.983855
Crop_Yield	0.714383	0.978614	0.983855	1.000000

Empty DataFrame

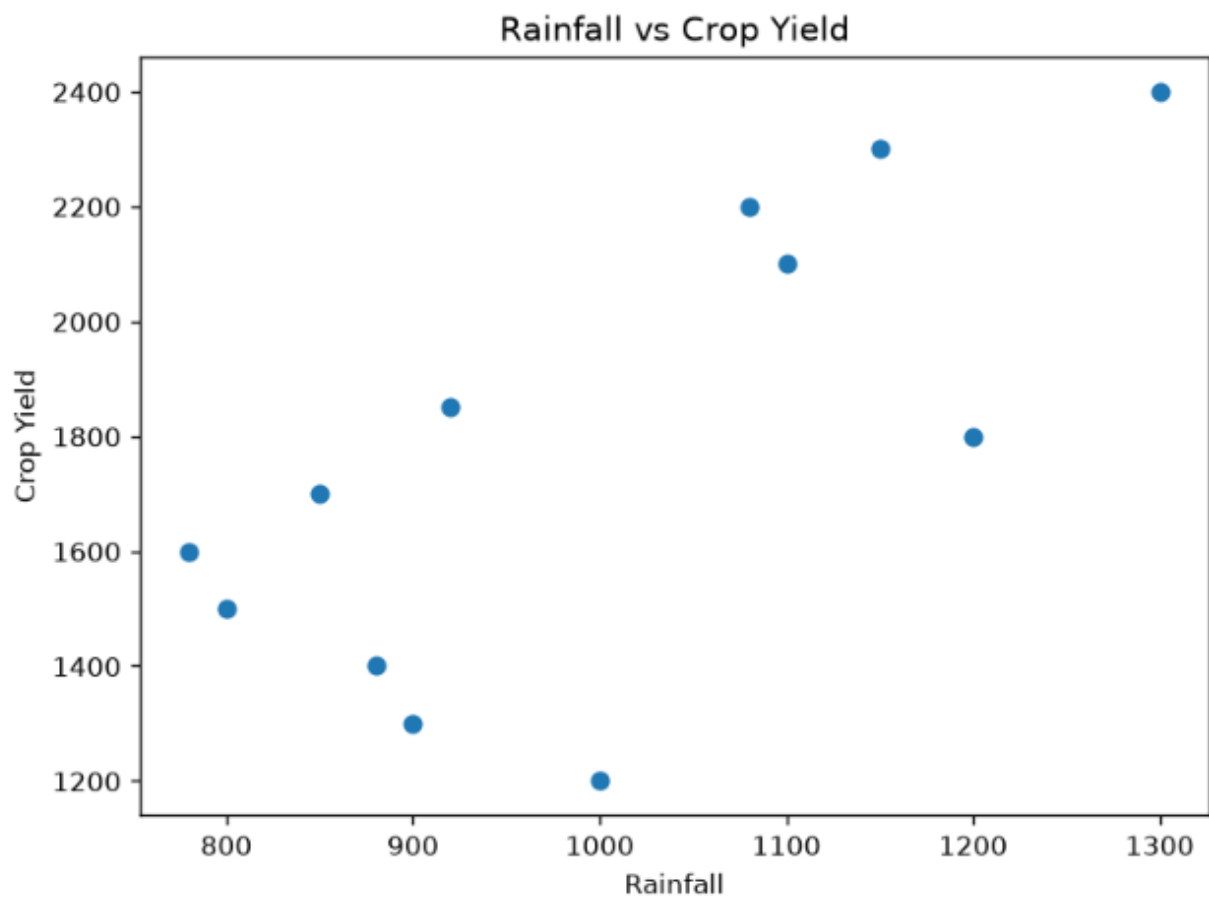
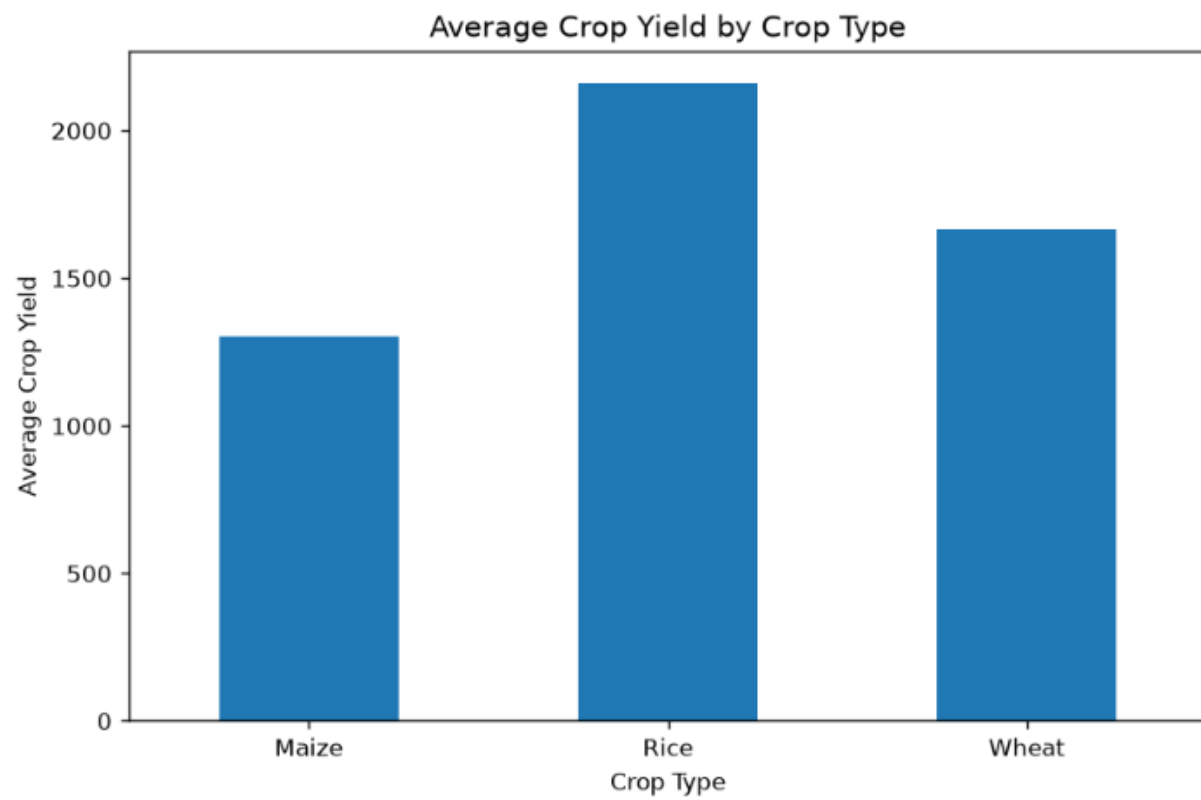
Columns: [Crop_Type, District, Cultivated_Area, Rainfall, Fertilizer_Usage, Crop_Yield]

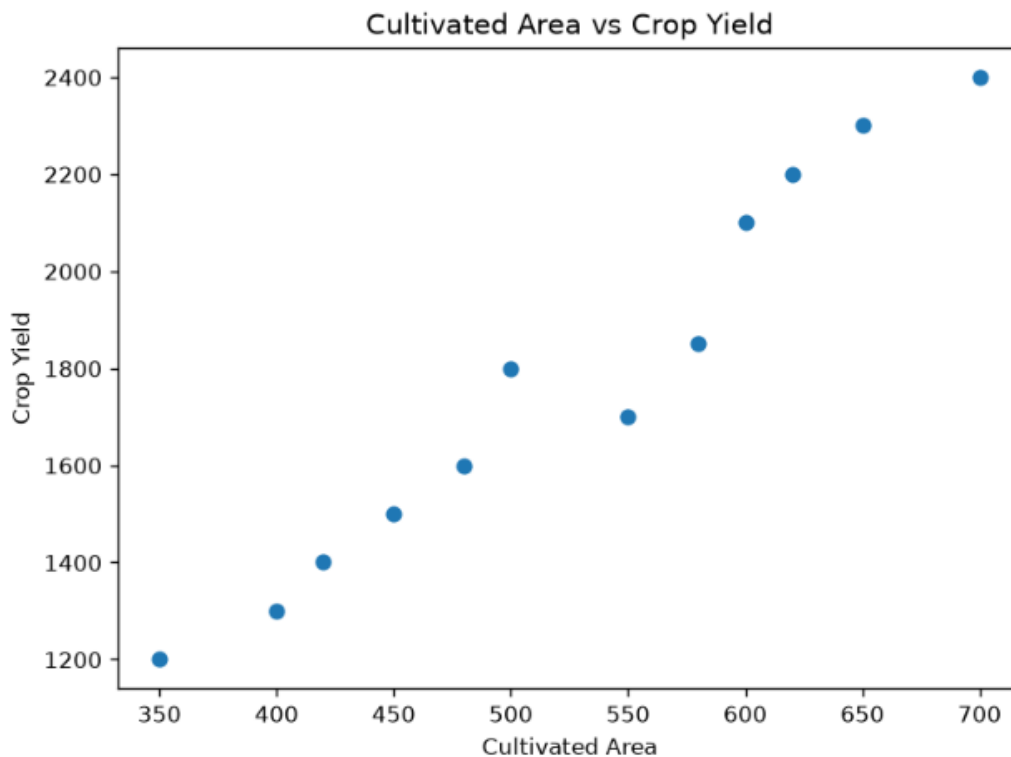
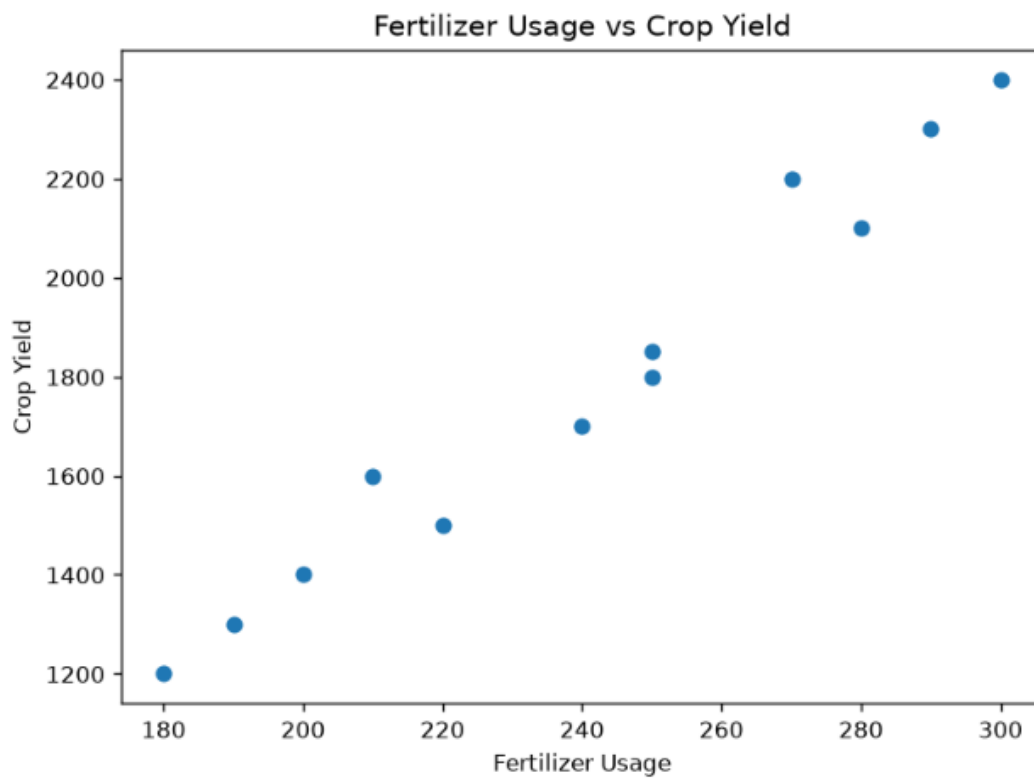
Index: []

Empty DataFrame

Columns: [Crop_Type, District, Cultivated_Area, Rainfall, Fertilizer_Usage, Crop_Yield]

Index: []





Result

The agriculture dataset was successfully explored using Pandas. Grouping, correlation analysis, outlier detection and visualization were used to identify factors associated with crop productivity. The analysis can support better fertilizer planning, crop selection and district-level agricultural decision-making.